

Технічне завдання:

28 жовтня 2024 р. 12:08

Попередні вимоги

1. тензо датчики до 50кг.
2. Необхідно виміряти вагу.
3. Прилад буде вимірювати вагу вулика з бджолами, він повинен виходити з режиму глибокого сна кожні тридцять хвилин, виміряти вагу вулика та підключатись до мережі WIFI та через протокол MQTT підключатись до серверу та передавати поточну вагу, та різницю з попереднім заміром. Також додатково до нього буде під'єднаний датчик температури та вологості DHT22, та разом з вагою він буде передавати данні щодо температури, та вологості.

Технічне завдання:

1. **Тип тензо-датчиків:** Тензо-датчики, які можуть вимірювати вагу до 50 кг.
2. **Параметри вимірювання:** Вимірювання ваги вулика з бджолами.
3. **Функціональність:**
 - Прокидання з режиму глибокого сну кожні 30 хвилин.
 - Вимірювання ваги вулика та передача даних через Wi-Fi та MQTT.
 - Відправка поточної ваги та різниці з попереднім заміром.
 - Підключення додаткового датчика температури та вологості DHT22.

Проектування моделі та структури класів:

1. **Клас Тензо-датчик (StrainGauge):**
 - Методи для зчитування ваги.
 - Методи для калібрування та обробки даних.
2. **Клас Контролер (Controller):**
 - Методи для керування режимом сну та періодичними вимірюваннями.
 - Методи для зчитування ваги та даних з датчика температури та вологості.
3. **Клас Wi-Fi та MQTT (WiFi_MQTT):**
 - Методи для підключення до мережі Wi-Fi та передачі даних через протокол MQTT.
4. **Клас Датчик температури та вологості (DHT22Sensor):**
 - Методи для зчитування даних про температуру та вологість.

Кодування:

1. **Реалізація класу Тензо-датчик (StrainGauge):**
 - Напишіть код для зчитування ваги з тензо-датчиків та їх калібрування.
2. **Реалізація класу Контролер (Controller):**
 - Напишіть код для керування режимом сну та періодичними вимірюваннями ваги.
3. **Реалізація класу Wi-Fi та MQTT (WiFi_MQTT):**
 - Напишіть код для підключення до мережі Wi-Fi та передачі даних через MQTT.
4. **Реалізація класу Датчик температури та вологості (DHT22Sensor):**
 - Напишіть код для зчитування даних про температуру та вологість з датчика DHT22.

Тестування:

1. **Перевірка роботи кожного класу окремо:**
 - Перевірте, чи правильно зчитуються дані з тензо-датчиків та датчика DHT22.
 - Перевірте, чи працює підключення до Wi-Fi та передача даних через MQTT.
 - Переконайтеся, що контролер коректно виконує свої функції управління.

2. Інтеграційне тестування:

- Перевірте взаємодію всіх класів разом.
- Переконайтеся, що дані відправляються та обробляються правильно.

3. Тестування на реальному обладнанні:

- Перевірте роботу програми на реальному пристрої ESP8266 з тензо-датчиками та датчиком DHT22.

Структура класів

28 жовтня 2024 р. 12:13

1. Клас StrainGauge (Тензо-датчик):

Властивості:

- `calibration_factor`: Коефіцієнт калібрування тензо-датчика.

Методи:

- `__init__(self, calibration_factor)`: Конструктор класу, приймає коефіцієнт калібрування.
- `measure_weight(self) -> float`: Метод для вимірювання ваги. Повертає значення ваги у кілограмах.
- `calibrate(self, known_weight: float)`: Метод для калібрування тензо-датчика. Приймає відому вагу в кілограмах та оновлює `calibration_factor`.

2. Клас Controller (Контролер):

Властивості:

- `sleep_interval`: Інтервал періодичного сну.

Методи:

- `__init__(self, sleep_interval)`: Конструктор класу, приймає інтервал періодичного сну.
- `enter_deep_sleep(self)`: Метод для входу в режим глибокого сну.
- `wake_up(self)`: Метод для прокидання з режиму глибокого сну.
- `measure_and_send_data(self)`: Метод для вимірювання ваги та інших даних та відправлення їх через Wi-Fi та MQTT.

3. Клас WiFi_MQTT (Wi-Fi та MQTT):

Властивості:

- `wifi_ssid`: SSID мережі Wi-Fi.
- `wifi_password`: Пароль мережі Wi-Fi.
- `mqtt_broker`: Адреса брокера MQTT.
- `mqtt_username`: Ім'я користувача для підключення до MQTT.
- `mqtt_password`: Пароль для підключення до MQTT.

Методи:

- `__init__(self, wifi_ssid, wifi_password, mqtt_broker, mqtt_username, mqtt_password)`: Конструктор класу, приймає дані для підключення до Wi-Fi та MQTT.
- `connect_to_wifi(self)`: Метод для підключення до мережі Wi-Fi.
- `connect_to_mqtt(self)`: Метод для підключення до брокера MQTT.
- `send_data(self, data)`: Метод для відправлення даних через MQTT.

4. Клас DHT22Sensor (Датчик температури та вологості):

Методи:

- `__init__(self)`: Конструктор класу.
- `read_temperature(self) -> float`: Метод для зчитування температури. Повертає значення температури у градусах Цельсія.
- `read_humidity(self) -> float`: Метод для зчитування вологості. Повертає значення вологості відносної відсотками.

Оновлене технічне завдання

1. **Мета проекту:** Розробити програму для ESP8266, яка буде керувати тензодатчиками до 50 кг через модуль HX711 AD, вимірювати вагу вулика з бджолами, а також збирати дані з датчика температури та вологості DHT22. Програма повинна враховувати стан датчика Холла для визначення відкриття вулика.
2. **Функціональні вимоги:**
 - Зчитування даних з тензодатчиків (використати бібліотеку <https://github.com/GyverLibs/GyverHX711>).
 - Зчитування даних з датчика DHT22.
 - Обробка та калібрування даних.
 - Передача даних на сервер через Wi-Fi та протокол MQTT.
 - Вихід з режиму глибокого сну кожні 30 хвилин для вимірювання ваги та збору даних.
 - Перевірка стану датчика Холла для визначення відкриття вулика.
 - Додаткова перевірка ваги для уникнення помилкових даних.
 - Зберігання даних у внутрішню енергонезалежну пам'ять (EEPROM) при відсутності зв'язку з сервером.
 - Завантаження даних з пам'яті на сервер при відновленні зв'язку.
 - Вихід з режиму глибокого сну по спрацюванню датчика Холла.
 - Передача на сервер інформації про відкриття вулика та встановлення параметру, що вулик відкривався, і час відкриття.
 - При наступному вимірюванні ваги, якщо параметр, що вулик відкривався, встановлений, значне відхилення ваги не вважається помилкою, і параметр про відкриття вулика скидається.
3. **Нефункціональні вимоги:**
 - Надійність та стабільність роботи.
 - Енергоефективність.
 - Легкість у налаштуванні та використанні.

Опис процесу роботи контролера

4. **Ініціалізація:**
 - Контролер виходить з режиму глибокого сну.
 - Ініціалізуються всі сенсори та модулі (тензодатчик, датчик DHT22, Wi-Fi, MQTT, RTC, EEPROM).
5. **Перевірка стану датчика Холла:**
 - Якщо датчик Холла спрацьовує (вулик відкритий), контролер записує факт відкриття вулика, передає цю інформацію на сервер і повертається в режим глибокого сну.
6. **Синхронізація часу:**
 - Контролер підключається до Wi-Fi і синхронізує час по NTP.
7. **Зчитування даних:**
 - Контролер зчитує вагу, температуру та вологість.
 - Якщо зміна ваги перевищує поріг, виконується додаткова перевірка ваги (три вимірювання з інтервалом).
8. **Обробка даних:**
 - Якщо вулик був відкритий, значне відхилення ваги не вважається помилкою.
 - Параметр про відкриття вулика скидається.
9. **Передача даних:**
 - Контролер передає дані на сервер через MQTT.
 - Якщо зв'язок з сервером відсутній, дані зберігаються в EEPROM.
10. **Завантаження даних з пам'яті:**
 - При відновленні зв'язку з сервером, контролер завантажує дані з EEPROM на сервер.
11. **Повернення в режим глибокого сну:**
 - Контролер повертається в режим глибокого сну на 30 хвилин або до спрацювання датчика Холла.

Опис логіки для уточнення реалізації:

датчика Холла:

виведення з режиму глибокого сну по спрацюванню датчика відкриття (встановити приривання), та передавання на сервер інформацію про відкриття а також встановлення параметру що вулик відкривався та час відкриття в EEPROM. При наступному вимірюванні ваги, якщо параметр що вулик відкривався встановлений, то значне відхилення ваги може бути нормою та не вважається помилкою, параметр про відкриття вулика скидається.

Зберігання даних в пам'ять:

логіка зберігання та вивантаження даних повинна бути такою: якщо немає зв'язку з сервером mqtt данні записуються в пам'ять та накопичуються там (наприклад за декілька днів поки немає доступу до інтернету), якщо зв'язок з'являється данні з пам'яті вивантажуються на сервер (крім першого запису) в тій же послідовності, пам'ять очищується, після цього на сервер вивантажуються данні останнього заміру та в пам'ять записуються данні останнього заміру.

Вимірювання ваги:

у зв'язку з тим, що датчики ваги часто видають хибні данні необхідно додати додаткову перевірку: якщо вага суттєво відрізняється від попередньої то потрібно зробити додаткових декілька вимірювань, щоб впевнитися у точності вимірювання. Якщо в період після останнього до поточного вимірювання вулик відкривався то значне відхилення ваги (може досягати 50 кг.) не вважається помилкою.

Для збору статистики таких помилкових даних потрібно додати признак помилки та передавати його на сервер разом з даними у випадку якщо значення WEIGHT_THRESHOLD було перевищено, якщо значення перевищено не було признак помилки буде мати признак false.

Оновлена структура класів

1. **Інтерфейс ISensor:**
 - **Методи:**
 - void begin(): Ініціалізація сенсора.
 - float read(): Зчитування даних з сенсора.
2. **Клас LoadCell (реалізує ISensor):**
 - **Властивості:**
 - GyverHX711 hx711: Об'єкт для роботи з модулем HX711.
 - **Методи:**
 - LoadCell(uint8_t dout, uint8_t sck): Конструктор, який приймає пін-коди для підключення до HX711.
 - void begin(): Ініціалізація HX711.
 - float read(): Зчитування ваги з тензодатчика.
3. **Клас DHTSensor (реалізує ISensor):**
 - **Властивості:**
 - DHT dht: Об'єкт для роботи з датчиком DHT.
 - **Методи:**
 - DHTSensor(uint8_t pin, uint8_t type): Конструктор, який приймає пін-код та тип датчика.
 - void begin(): Ініціалізація датчика DHT.
 - float readTemperature(): Зчитування температури.
 - float readHumidity(): Зчитування вологості.
 - float read(): Зчитування даних (температури або вологості).
4. **Інтерфейс ICommunication:**
 - **Методи:**
 - void connect(): Підключення до мережі.
 - void sendData(const char* topic, const char* payload): Відправка даних.

5. Клас WiFiManager (реалізує ICommunication):

- **Властивості:**
- const char* ssid: SSID Wi-Fi мережі.
- const char* password: Пароль Wi-Fi мережі.
- **Методи:**
- WiFiManager(const char* ssid, const char* password): Конструктор, який приймає SSID та пароль.
- void connect(): Підключення до Wi-Fi мережі.
- void sendData(const char* topic, const char* payload): Відправка даних через Wi-Fi.

6. Клас MQTTClient (реалізує ICommunication):

- **Властивості:**
- const char* mqtt_server: Адреса MQTT сервера.
- int mqtt_port: Порт MQTT сервера.
- const char* mqtt_user: Ім'я користувача для MQTT.
- const char* mqtt_password: Пароль для MQTT.
- WiFiClient espClient: Об'єкт для роботи з Wi-Fi.
- PubSubClient client: Об'єкт для роботи з MQTT.
- **Методи:**
- MQTTClient(const char* server, int port, const char* user, const char* password): Конструктор, який приймає параметри для підключення до MQTT сервера.
- void begin(): Ініціалізація MQTT клієнта.
- void connect(): Підключення до MQTT сервера.
- void sendData(const char* topic, const char* payload): Відправка даних на MQTT сервер.

7. Клас MemoryManager:

- **Властивості:**
- const int maxRecords: Максимальна кількість записів.
- int recordSize: Розмір одного запису.
- **Методи:**
- void begin(): Ініціалізація EEPROM.
- void saveData(const DataRecord& data): Збереження даних у EEPROM.
- bool readData(DataRecord& data, int index): Зчитування даних з EEPROM за індексом.
- bool readLastData(DataRecord& data): Зчитування останнього запису з EEPROM.
- void clearData(int index): Очищення даних у EEPROM за індексом.
- int getRecordCount(): Отримання кількості записів у EEPROM.

8. Клас RTCManager:

- **Властивості:**
- iarduino_RTC rtc: Об'єкт для роботи з RTC.
- WiFiUDP ntpUDP: Об'єкт для роботи з NTP.
- NTPClient timeClient: Об'єкт для роботи з NTP клієнтом.
- **Методи:**
- void begin(): Ініціалізація RTC та NTP клієнта.

Вы

Отправлено пользователем вы:

ти не закінчив

Copilot

Отправлено пользователем Copilot:

Дякую за нагадування! Давайте завершимо опис методів та структури класів.

Оновлена структура класів (продовження)

8. Клас RTCManager:

- **Властивості:**
- iarduino_RTC rtc: Об'єкт для роботи з RTC.
- WiFiUDP ntpUDP: Об'єкт для роботи з NTP.
- NTPClient timeClient: Об'єкт для роботи з NTP клієнтом.
- **Методи:**
- void begin(): Ініціалізація RTC та NTP клієнта.
- void syncTime(): Синхронізація часу по NTP.

- String getTimestamp(): Отримання поточного часу у вигляді рядка.

Опис методів та параметрів

1. Методи інтерфейсу ISensor:

- void begin(): Ініціалізує сенсор, готує його до роботи.
- float read(): Зчитує дані з сенсора та повертає їх у вигляді числа з плаваючою комою.

2. Методи класу LoadCell:

- LoadCell(uint8_t dout, uint8_t sck): Конструктор, який приймає пін-коди для підключення до HX711.
- void begin(): Ініціалізує HX711, встановлює масштаб та обнуляє вагу.
- float read(): Зчитує вагу з тензодатчика та повертає її у вигляді числа з плаваючою комою.

3. Методи класу DHTSensor:

- DHTSensor(uint8_t pin, uint8_t type): Конструктор, який приймає пін-код та тип датчика.
- void begin(): Ініціалізує датчик DHT.
- float readTemperature(): Зчитує температуру з датчика та повертає її у вигляді числа з плаваючою комою.
- float readHumidity(): Зчитує вологість з датчика та повертає її у вигляді числа з плаваючою комою.
- float read(): Зчитує дані (температури або вологості) та повертає їх у вигляді числа з плаваючою комою.

4. Методи інтерфейсу ICommunication:

- void connect(): Підключає пристрій до мережі.
- void sendData(const char* topic, const char* payload): Відправляє дані на вказану тему.

5. Методи класу WiFiManager:

- WiFiManager(const char* ssid, const char* password): Конструктор, який приймає SSID та пароль для підключення до Wi-Fi мережі.
- void connect(): Підключає пристрій до Wi-Fi мережі.
- void sendData(const char* topic, const char* payload): Відправляє дані через Wi-Fi.

6. Методи класу MQTTClient:

- MQTTClient(const char* server, int port, const char* user, const char* password): Конструктор, який приймає параметри для підключення до MQTT сервера.
- void begin(): Ініціалізує MQTT клієнта.
- void connect(): Підключає пристрій до MQTT сервера.
- void sendData(const char* topic, const char* payload): Відправляє дані на MQTT сервер.

7. Методи класу MemoryManager:

- void begin(): Ініціалізує EEPROM.
- void saveData(const DataRecord& data): Зберігає дані у EEPROM.
- bool readData(DataRecord& data, int index): Зчитує дані з EEPROM за індексом.
- bool readLastData(DataRecord& data): Зчитує останній запис з EEPROM.
- void clearData(int index): Очищує дані у EEPROM за індексом.
- int getRecordCount(): Отримує кількість записів у EEPROM.

8. Методи класу RTCManager:

- void begin(): Ініціалізує RTC та NTP клієнта.
- void syncTime(): Синхронізує час по NTP.
- String getTimestamp(): Отримує поточний час у вигляді рядка.

використати бібліотеку <https://github.com/GyverLibs/GyverHX711>

Структура проекту

30 жовтня 2024 р. 8:34

```
/src
/sensors
  ISensor.h
  LoadCell.h
  LoadCell.cpp
  DHTSensor.h
  DHTSensor.cpp
/communication
  ICommunication.h
  WiFiManager.h
  WiFiManager.cpp
  MQTTClient.h
  MQTTClient.cpp
/storage
  MemoryManager.h
  MemoryManager.cpp
/rtc
  RTCManager.h
  RTCManager.cpp
main.cpp
```


ISensor.h

30 жовтня 2024 р. 8:40

```
#ifndef ISENSOR_H
#define ISENSOR_H

class ISensor {
public:
    virtual void begin() = 0;
    virtual float read() = 0;
};

#endif // ISENSOR_H
```

LoadCell.h

30 жовтня 2024 р. 8:41

```
#ifndef LOADCELL_H
#define LOADCELL_H

#include "ISensor.h"
#include <GyverHX711.h>

class LoadCell : public ISensor {
public:
    LoadCell(uint8_t dout, uint8_t sck);
    void begin() override;
    float read() override;
private:
    GyverHX711 hx711;
};

#endif // LOADCELL_H
```

LoadCell.cpp

30 жовтня 2024 р. 8:41

```
#include "LoadCell.h"
```

```
LoadCell::LoadCell(uint8_t dout, uint8_t sck) : hx711(dout, sck) {}
```

```
void LoadCell::begin() {  
    hx711.begin();  
    hx711.setScale(2280.f); // Встановлюємо масштаб (потрібно калібрувати)  
    hx711.tare(); // Обнулення ваги  
}
```

```
float LoadCell::read() {  
    return hx711.getUnits(10); // Зчитування ваги  
}
```

DHTSensor.h

30 жовтня 2024 р. 8:42

```
#ifndef DHTSENSOR_H
#define DHTSENSOR_H

#include "ISensor.h"
#include <DHT.h>

class DHTSensor : public ISensor {
public:
    DHTSensor(uint8_t pin, uint8_t type);
    void begin() override;
    float readTemperature();
    float readHumidity();
    float read() override;
private:
    DHT dht;
};

#endif // DHTSENSOR_H
```

DHTSensor.cpp

30 жовтня 2024 р. 8:43

```
#include "DHTSensor.h"

DHTSensor::DHTSensor(uint8_t pin, uint8_t type) : dht(pin, type) {}

void DHTSensor::begin() {
    dht.begin();
}

float DHTSensor::readTemperature() {
    return dht.readTemperature();
}

float DHTSensor::readHumidity() {
    return dht.readHumidity();
}

float DHTSensor::read() {
    return readTemperature(); // Можна змінити на readHumidity() або інший метод
}
```

ICommunication.h

30 жовтня 2024 р. 8:43

```
#ifndef ICOMMUNICATION_H
#define ICOMMUNICATION_H

class ICommunication {
public:
    virtual void connect() = 0;
    virtual void sendData(const char* topic, const char* payload) = 0;
};

#endif // ICOMMUNICATION_H
```

WiFiManager.h

30 жовтня 2024 р. 8:44

```
#ifndef WIFIMANAGER_H
#define WIFIMANAGER_H

#include "ICommunication.h"
#include <ESP8266WiFi.h>

class WiFiManager : public ICommunication {
public:
    WiFiManager(const char* ssid, const char* password);
    void connect() override;
    void sendData(const char* topic, const char* payload) override;
private:
    const char* ssid;
    const char* password;
};

#endif // WIFIMANAGER_H
```

WiFiManager.cpp

30 жовтня 2024 р. 8:44

```
#include "WiFiManager.h"
```

```
WiFiManager::WiFiManager(const char* ssid, const char* password) : ssid(ssid),  
password(password) {}
```

```
void WiFiManager::connect() {  
    WiFi.begin(ssid, password);  
    while (WiFi.status() != WL_CONNECTED) {  
        delay(1000);  
    }  
}
```

```
void WiFiManager::sendData(const char* topic, const char* payload) {  
    // Реалізація відправки даних через Wi-Fi  
}
```


MQTTClient.h

30 жовтня 2024 р. 8:44

```
#ifndef MQTTCLIENT_H
#define MQTTCLIENT_H

#include "ICommunication.h"
#include <ESP8266WiFi.h>
#include <PubSubClient.h>

class MQTTClient : public ICommunication {
public:
    MQTTClient(const char* server, int port, const char* user, const char* password);
    void begin();
    void connect() override;
    void sendData(const char* topic, const char* payload) override;
private:
    const char* mqtt_server;
    int mqtt_port;
    const char* mqtt_user;
    const char* mqtt_password;
    WiFiClient espClient;
    PubSubClient client{espClient};
};

#endif // MQTTCLIENT_H
```

MQTTClient.cpp

30 жовтня 2024 р. 8:44

```
#include "MQTTClient.h"
```

```
MQTTClient::MQTTClient(const char* server, int port, const char* user, const char* password)  
    : mqtt_server(server), mqtt_port(port), mqtt_user(user), mqtt_password(password) {}
```

```
void MQTTClient::begin() {  
    client.setServer(mqtt_server, mqtt_port);  
}
```

```
void MQTTClient::connect() {  
    while (!client.connected()) {  
        client.connect("ESP8266Client", mqtt_user, mqtt_password);  
        delay(1000);  
    }  
}
```

```
void MQTTClient::sendData(const char* topic, const char* payload) {  
    client.publish(topic, payload);  
}
```

MemoryManager.h

30 жовтня 2024 р. 9:06

```
#ifndef MEMORYMANAGER_H
#define MEMORYMANAGER_H

#include <EEPROM.h>

struct DataRecord {
    char timestamp[20];
    float weight;
    float temperature;
    float humidity;
    float weightChange;
    bool errorFlag; // Признак помилки
};

class MemoryManager {
public:
    void begin();
    void saveData(const DataRecord& data);
    bool readData(DataRecord& data, int index);
    void clearData(int index);
    int getRecordCount();
private:
    const int maxRecords = 10; // Максимальна кількість записів
    int recordSize = sizeof(DataRecord);
};

#endif // MEMORYMANAGER_H
```

MemoryManager.cpp

30 жовтня 2024 р. 9:09

```
#include "MemoryManager.h"

void MemoryManager::begin() {
    EEPROM.begin(512);
}

void MemoryManager::saveData(const DataRecord& data) {
    int recordCount = getRecordCount();
    if (recordCount < maxRecords) {
        EEPROM.put(recordCount * recordSize, data);
        EEPROM.commit();
    }
}

bool MemoryManager::readData(DataRecord& data, int index) {
    if (index < maxRecords) {
        EEPROM.get(index * recordSize, data);
        return true;
    }
    return false;
}

void MemoryManager::clearData(int index) {
    if (index < maxRecords) {
        DataRecord emptyData = {};
        EEPROM.put(index * recordSize, emptyData);
        EEPROM.commit();
    }
}

int MemoryManager::getRecordCount() {
    int count = 0;
    DataRecord data;
    for (int i = 0; i < maxRecords; i++) {
        EEPROM.get(i * recordSize, data);
        if (data.timestamp[0] != '\0') {
            count++;
        }
    }
    return count;
}
```

RTCManager.h

30 жовтня 2024 р. 9:33

```
#ifndef RTCMANAGER_H
#define RTCMANAGER_H

#include <iarduino_RTC.h>

class RTCManager {
public:
    void begin();
    String getTimestamp();
private:
    iarduino_RTC rtc;
};

#endif // RTCMANAGER_H
```

RTCManager.cpp

30 жовтня 2024 р. 9:34

```
#include "RTCManager.h"
```

```
void RTCManager::begin() {  
    rtc.begin();  
}
```

```
String RTCManager::getTimestamp() {  
    rtc.gettime();  
    char timestamp[20];  
    sprintf(timestamp, sizeof(timestamp), "%04d-%02d-%02d %02d:%02d:%02d", rtc.year,  
rtc.month, rtc.day, rtc.hour, rtc.minute, rtc.second);  
    return String(timestamp);  
}
```

```
#include "sensors/LoadCell.h"
#include "sensors/DHTSensor.h"
#include "communication/WiFiManager.h"
#include "communication/MQTTClient.h"
#include "storage/MemoryManager.h"
#include "rtc/RTCManager.h"

#define DHTPIN D4
#define DHTTYPE DHT22
#define WEIGHT_THRESHOLD 5.0 // Поріг зміни ваги для додаткової перевірки
#define HALL_SENSOR_PIN D5 // Пін для датчика Холла
#define SIGNIFICANT_WEIGHT_CHANGE 20.0 // Допустима значна зміна ваги при відкритті вулика

LoadCell loadCell(D2, D3);
DHTSensor dhtSensor(DHTPIN, DHTTYPE);
WiFiManager wifiManager("yourSSID", "yourPASSWORD");
MQTTClient mqttClient("yourMQTTServer", 1883, "yourMQTTUser", "yourMQTTPassword");
MemoryManager memoryManager;
RTCManager rtcManager;

float getAccurateWeight() {
    float weight1 = loadCell.read();
    delay(1000);
    float weight2 = loadCell.read();
    delay(1000);
    float weight3 = loadCell.read();

    // Повертаємо середнє значення трьох вимірювань
    return (weight1 + weight2 + weight3) / 3.0;
}

void uploadStoredData() {
    int recordCount = memoryManager.getRecordCount();
    for (int i = 1; i < recordCount; i++) {
        DataRecord data;
        if (memoryManager.readData(data, i)) {
            char payload[250];
            sprintf(payload, sizeof(payload), "Timestamp: %s, Weight: %.2f kg, Temp: %.2f C, Humidity: %.2f %%, Weight Change: %.2f kg, Error: %s",
                data.timestamp, data.weight, data.temperature, data.humidity, data.weightChange,
                data.errorFlag ? "true" : "false");
            mqttClient.sendData("hive/data", payload);
            memoryManager.clearData(i);
        }
    }
}

void wakeUp() {
    loadCell.begin();
    dhtSensor.begin();
    memoryManager.begin();
}
```

```

rtcManager.begin();

wifiManager.connect();
rtcManager.syncTime();

// Перевірка стану датчика Холла
bool hiveOpened = digitalRead(HALL_SENSOR_PIN) == LOW;

if (hiveOpened) {
    Serial.println("Hive is opened. Skipping measurements.");
    return;
}

float weight = getAccurateWeight();
float temperature = dhtSensor.readTemperature();
float humidity = dhtSensor.readHumidity();

DataRecord lastData;
memoryManager.readData(lastData, 0);

float weightChange = weight - lastData.weight;
bool errorFlag = false;

// Додаткова перевірка ваги
if (abs(weightChange) > WEIGHT_THRESHOLD) {
    weight = getAccurateWeight();
    weightChange = weight - lastData.weight;
    errorFlag = true;
}

// Якщо вулик був відкритий, допускається значна зміна ваги
if (hiveOpened && abs(weightChange) > SIGNIFICANT_WEIGHT_CHANGE) {
    errorFlag = false;
}

DataRecord newData;
String timestamp = rtcManager.getTimestamp();
strncpy(newData.timestamp, timestamp.c_str(), sizeof(newData.timestamp));
newData.weight = weight;
newData.temperature = temperature;
newData.humidity = humidity;
newData.weightChange = weightChange;
newData.errorFlag = errorFlag;

char payload[250];
snprintf(payload, sizeof(payload), "Timestamp: %s, Weight: %.2f kg, Temp: %.2f C, Humidity: %.2f
%%, Weight Change: %.2f kg, Error: %s",
        newData.timestamp, weight, temperature, humidity, weightChange, errorFlag ? "true" :
"false");

mqttClient.begin();
mqttClient.connect();

if (!mqttClient.client.connected()) {
    memoryManager.saveData(newData);
} else {
    uploadStoredData();
}

```



```

        mqttClient.sendData("hive/data", payload);
        memoryManager.clearData(0);
        memoryManager.saveData(newData);
    }
}

void setup() {
    Serial.begin(115200);
    EEPROM.begin(512);
    pinMode(HALL_SENSOR_PIN, INPUT_PULLUP); // Налаштування піну для датчика Холла
}

void loop() {
    wakeUp();
    ESP.deepSleep(30 * 60 * 1000000); // Сплячий режим на 30 хвилин
    delay(100); // Затримка для стабілізації
}

```